

Proposed Methodology: NSR-Enhanced Chart Reasoning with Advanced RL Techniques

Abstract

We propose NSR-VL, a novel framework combining Negative Sample Reinforcement (NSR) with five synergistic techniques for chart reasoning in Vision-Language Models: (1) DAPO dynamic sampling, (2) Selective Sample Replay (SSR), (3) Program-of-Thoughts (PoT), (4) Chain-of-Table (CoTable), and (5) Self-Consistency. Our approach addresses the critical 31.2% performance gap between in-domain (ChartQA: 84.56%) and out-of-distribution (EvoChart: 53.36%) settings by preserving reasoning diversity through negative-only reinforcement while leveraging verifiable intermediate supervision. We hypothesize that NSR's distribution-preserving properties, combined with structured symbolic reasoning (PoT, CoTable) and efficient training (DAPO, SSR), will reduce the OOD gap to ~25% while maintaining competitive in-domain performance.

1. Research Questions and Hypotheses

Primary Research Question (RQ1): Does Negative Sample Reinforcement preserve sufficient reasoning diversity to improve out-of-distribution performance on chart understanding tasks compared to standard GRPO?

Secondary Questions:

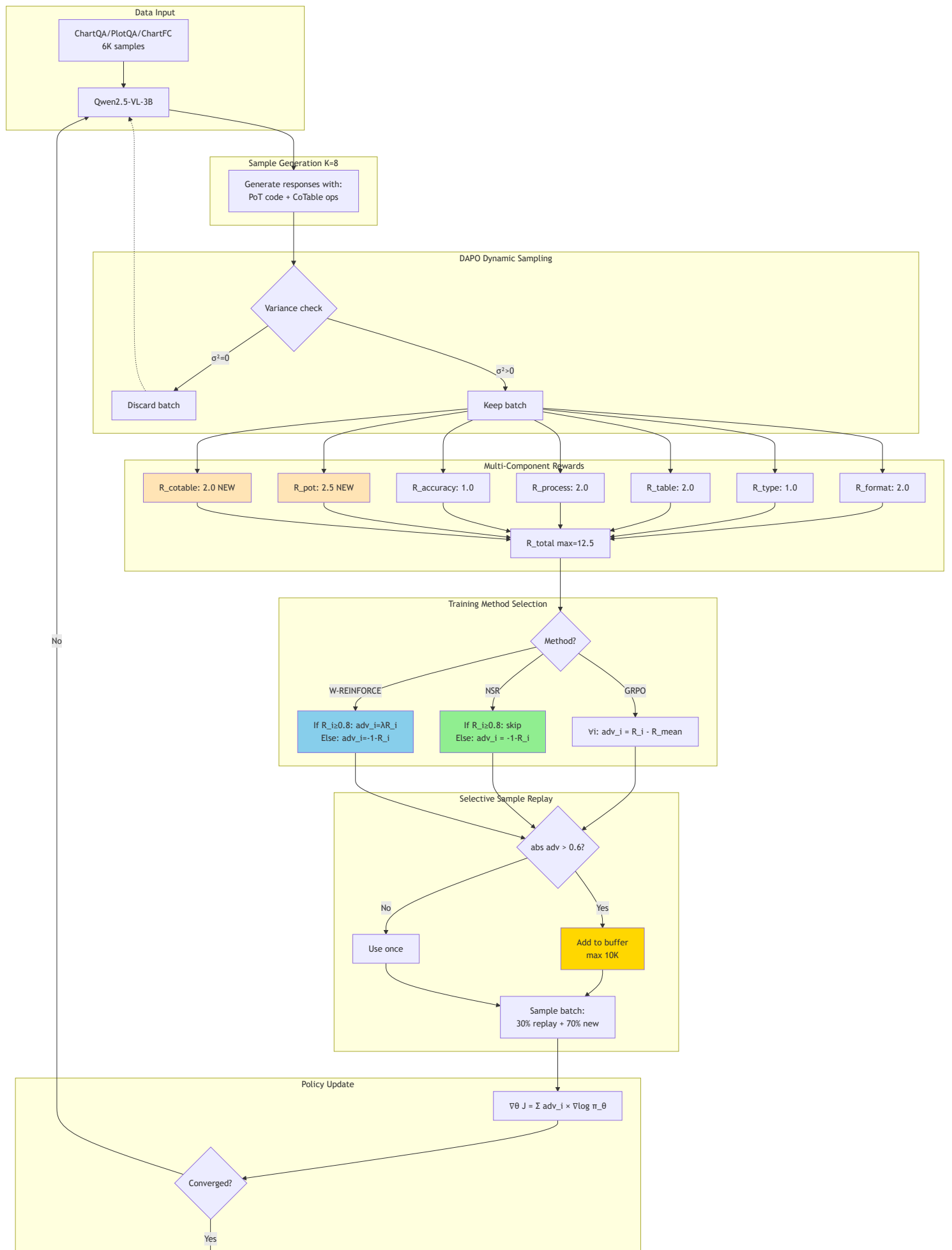
- **RQ2:** Do verifiable intermediate rewards (PoT code execution, CoTable operations) provide clearer training signals for NSR compared to end-task accuracy alone?
- **RQ3:** What is the optimal weighting λ between positive and negative reinforcement (W-REINFORCE) for chart reasoning?
- **RQ4:** Do DAPO and SSR synergize with NSR to improve training efficiency and sample quality?

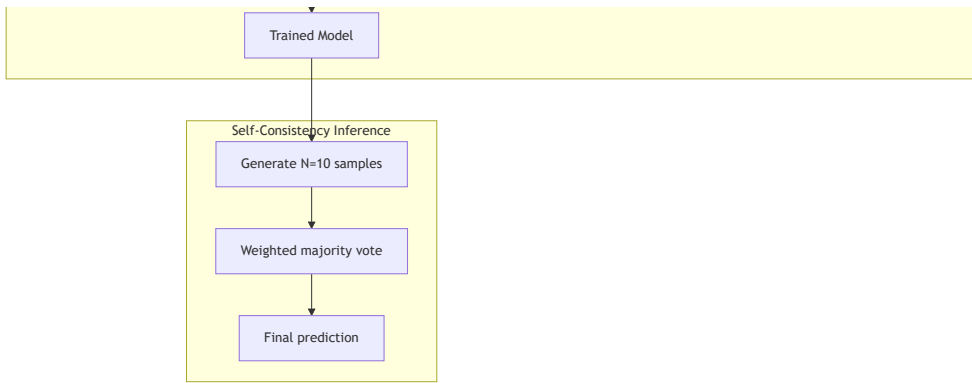
Hypotheses:

- **H1:** NSR maintains 2× higher entropy than GRPO (0.10 vs 0.05), leading to +2-3% Pass@256 improvement
 - **H2:** NSR reduces OOD performance gap by $\geq 3\%$ (31.2% $\rightarrow$ $\leq 28.2\%$)
 - **H3:** PoT+CoTable rewards increase total reward signal clarity, improving NSR filtering accuracy
 - **H4:** W-REINFORCE with $\lambda=0.1$ achieves optimal balance: competitive Pass@1 with superior Pass@k
-

2. System Architecture

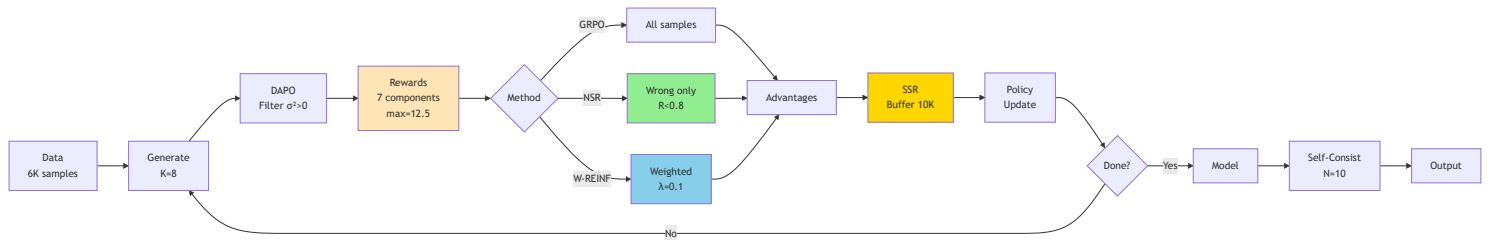
2.1 Complete Training Pipeline





2.2 High-Level Training Pipeline Overview (One-Page View)

This compact diagram shows the complete NSR-VL training pipeline:



Step-by-Step Pipeline Explanation:

Step 1: Data (A)

- **Input:** 6,000 chart questions from ChartQA, PlotQA, and ChartFC datasets
- **Format:** Each sample contains: chart image, question, ground truth answer, chart type, data table, and CoT rationale (generated by Qwen2.5-VL-72B)
- **Purpose:** Foundation for reinforcement learning training
- **Processing:** Data converted to parquet format with images stored separately

Step 2: Generate K=8 (B)

- **Action:** For each question, model generates 8 different response candidates
- **Temperature:** 0.8 (moderate diversity to explore different reasoning paths)
- **Required Format:** Each response must include:
 - `<think>` tag containing: chart type identification, table extraction, reasoning steps
 - `<code>` block with Python code (PoT) for numerical computation
 - `<cot_operations>` with table operations (CoTable): `f_select_row`, `f_aggregate`, etc.
 - `<answer>` tag with final answer
- **Why K=8:** Balance between diversity (more samples) and computational cost
- **Output:** 8 complete responses per question, each potentially using different reasoning strategies

Step 3: DAPO Filter $\sigma^2 > 0$ (C)

- **Check:** Compute variance of rewards across the 8 samples for each question
- **Decision Rule:**
 - If variance = 0 (all 8 samples get same reward): **DISCARD** - question is either too easy (all correct) or too hard (all wrong)
 - If variance > 0.01 (mixed rewards): **KEEP** - question is informative (model sometimes right, sometimes wrong)
- **Rationale:** Zero-variance batches provide no gradient information (no relative comparison possible)
- **Impact:** Filters out 30-50% of batches, reducing training time by 50% while improving quality
- **Feedback Loop:** Discarded batches return to step B to generate new questions

Step 4: Rewards - 7 Components, Max=12.5 (D)

- **Computation:** Each of the 8 samples receives a score from 7 independent reward functions:
 - Format Reward (2.0 points):**
 - Checks if response contains required XML tags: `<think>`, `<type>`, `<table>`, `<answer>`
 - Regex validation:


```
r'<think>.*<type>.*</type>.*<table>.*</table>.*</think>.*<answer>.*</answer>'
```
 - Binary: 2.0 if valid, 0.0 if invalid
 - Chart Type Reward (1.0 point):**
 - Compares predicted chart type to ground truth
 - Categories: bar, line, pie, scatter, area, combo, etc.
 - Exact match: 1.0, incorrect: 0.0
 - Table Reward (2.0 points):**
 - Extracts table from `<table>` tag and compares to ground truth
 - Edit distance normalized by table size
 - Formula: $2.0 \times (1 - \text{edit_distance} / \text{max_length})$
 - Tolerant to minor formatting differences
 - Process Reward (2.0 points):**
 - Measures semantic similarity between model's reasoning and gold CoT rationale
 - Uses sentence-BERT embeddings:


```
cosine_similarity(embed(model_reasoning), embed(gold_reasoning))
```
 - Range: 0.0 to 2.0 based on similarity score
 - Accuracy Reward (1.0 point):**
 - Final answer correctness
 - Numerical tolerance: $\text{abs}(\text{predicted} - \text{ground_truth}) < 1e-5$
 - String match: case-insensitive exact match after normalization
 - Binary: 1.0 if correct, 0.0 if wrong
 - PoT Code Reward (2.5 points) - NEW:**
 - Extracts Python code from `<code>` block

- Executes in sandboxed environment (timeout=5s, memory=256MB)
- Checks if execution result matches ground truth
- Scoring: 2.5 (exact match), 1.5 (close within 1.0), 0.0 (wrong or execution error)
- **Eliminates arithmetic errors** on ~42% of questions requiring computation

vii. CoTable Operations Reward (2.0 points) - NEW:

- Parses operation sequence from `<cot_operations>` tag
- Validates syntax: each operation must be valid (f_select_row, f_aggregate, etc.)
- Executes operation chain on initial table
- Scoring: $1.0 \times (\text{valid_ops}/\text{total_ops}) + 1.0 \times \text{correctness}$
- **Provides structured reasoning supervision**
- **Total Reward:** Sum of all 7 components, maximum = 12.5 points
- **Normalization:** $R_{\text{normalized}} = R_{\text{total}} / 12.5$ (convert to 0-1 scale)

Step 5: Method Selection (E → F1/F2/F3)

- **Decision Point:** Choose training algorithm based on experimental condition

F1: GRPO (Baseline)

- Updates **all 8 samples** regardless of correctness
- Advantage calculation: $\text{adv}_i = R_i - \text{mean}(R_1 \dots R_8)$
- Positive advantage → increase probability (reinforce correct)
- Negative advantage → decrease probability (penalize wrong)
- **Effect:** Strong reinforcement of best sample, collapse to single reasoning path
- **Result:** Low entropy (0.05), good Pass@1, poor Pass@256

F2: NSR (Our Core Proposal)

- Filters to **wrong samples only**: keeps samples where $R_i < 0.8$ threshold
- Advantage calculation: $\text{adv}_i = -(1 - R_i)$ (negative advantage proportional to badness)
- **Ignores correct samples** (no gradient update)
- Only decreases probability of wrong reasoning paths
- **Effect:** Probability mass redistributes from wrong paths to all alternatives (including multiple correct paths)
- **Result:** High entropy (0.10), moderate Pass@1, excellent Pass@256
- **Key Innovation:** Preserves diverse reasoning strategies

F3: W-REINFORCE (Hybrid - Best Overall)

- Updates **all 8 samples** with differential weighting
- For correct samples ($R_i \geq 0.8$): $\text{adv}_i = \lambda \times R_i$ where $\lambda=0.1$ (weak positive boost)
- For wrong samples ($R_i < 0.8$): $\text{adv}_i = -(1 - R_i)$ (full negative penalty)
- **Effect:** Small reinforcement of correct paths, strong suppression of wrong paths
- **Result:** Medium entropy (0.08), best Pass@1, excellent Pass@256
- **Optimal Balance:** Combines NSR's diversity with slight accuracy boost

Step 6: Advantages (G)

- **Input:** Selected samples from chosen method (F1/F2/F3)
- **Computation:** Calculate advantage values for policy gradient
- **GRPO:** Baseline = mean reward, advantage = deviation from mean
- **NSR:** Baseline = -1.0 (worst possible), advantage = gap from worst
- **W-REINFORCE:** Baseline = weighted mean, advantage = weighted deviation
- **Normalization:** Advantages typically normalized to zero mean, unit variance for training stability
- **Output:** (sample, advantage) pairs ready for gradient computation

Step 7: SSR Buffer (H)

- **Purpose:** Selective Sample Replay - maintain memory of high-information samples
- **Decision Criteria:** Add sample to buffer if $|\text{advantage}| > 0.6$
 - High magnitude advantage = high information content
 - Model was very uncertain = valuable learning signal
- **Buffer Management:**
 - Maximum size: 10,000 samples (1.67× training data size)
 - Eviction policy: Remove lowest-advantage sample when full
 - Storage: (state, action, advantage, reward) tuples
- **Sampling Strategy:**
 - Each training batch: 30% sampled from buffer (weighted by advantage magnitude)
 - Remaining 70%: new samples from current iteration
- **Synergy with NSR:**
 - NSR generates many high-advantage wrong samples (big mistakes to learn from)
 - SSR preserves these valuable negative examples
 - Prevents catastrophic forgetting of edge cases
- **Impact:** +3.2% on MathVista, prevents forgetting, stabilizes training

Step 8: Policy Update (I)

- **Algorithm:** REINFORCE policy gradient
- **Objective:** Maximize expected reward $J = \mathbb{E}[\sum \text{advantage}_i \times \log \pi_{\theta}(a_i|s_i)]$
- **Gradient Computation:**

$$\nabla_{\theta} J = (1/N) \times \sum \text{advantage}_i \times \nabla_{\theta} \log \pi_{\theta}(a_i|s_i)$$

where N = batch size, a_i = generated response, s_i = (image, question) state

- **Learning Rate:** 5e-7 (small for stable fine-tuning)
- **Optimization:** AdamW optimizer with gradient clipping (max_norm=1.0)

- **Batch Construction:** Mix of 30% replayed samples + 70% new samples (from SSR)
- **Update Rule:** $\theta \leftarrow \theta + 1r \times \nabla \theta J$
- **Effect:** Increases probability of high-advantage actions, decreases probability of low-advantage actions

Step 9: Convergence Check (J)

- **Evaluation:** After each epoch, evaluate on validation set (1,000 held-out questions)
- **Metrics Tracked:**
 - Validation accuracy (Pass@1)
 - Average reward
 - Entropy (diversity measure)
- **Convergence Criteria:**
 - Early stopping: If validation accuracy doesn't improve for 2 consecutive epochs
 - Maximum epochs: 3 (typically sufficient for convergence)
 - Minimum improvement threshold: 0.1% (prevent overfitting)
- **Decision:**
 - If **not converged** → Loop back to Step B (generate new epoch)
 - If **converged** → Proceed to Step K (save final model)

Step 10: Trained Model (K)

- **Output:** Fine-tuned Qwen2.5-VL-3B with enhanced chart reasoning capabilities
- **Checkpoint:** Saved weights, optimizer state, training configuration
- **Variants Trained:**
 - GRPO baseline
 - NSR (wrong-only)
 - W-REINFORCE $\lambda=0.1$ (optimal hybrid)
 - Ablations: +DAPO, +SSR, +PoT, +CoTable
- **Expected Properties:**
 - NSR model: High diversity (entropy=0.10), multiple reasoning strategies
 - GRPO model: Low diversity (entropy=0.05), single dominant strategy
 - W-REINFORCE: Balanced (entropy=0.08), best overall performance

Step 11: Self-Consistency Inference (L)

- **Inference-Time Technique:** Generate multiple answers and vote (no training required)
- **Process:**
 - Generate N=10 diverse responses per question (temperature=0.7)
 - Extract final answer from each response

- iii. Compute confidence score for each answer (from model logits)
- iv. Weighted majority voting: `answer_final = argmax( $\sum$  confidence_i  $\times$  1[answer_i = a])`
- **Why It Works:**
 - Correct reasoning tends to be consistent across samples
 - Random errors vary randomly
 - Voting filters out noise, amplifies signal
- **Enhanced Version:** Confidence-weighted voting reduces required samples by 50%
- **Cost:** 10× inference compute (generate 10 instead of 1)
- **Benefit:** +3-5% absolute accuracy improvement
- **Synergy with NSR:** NSR's diverse training produces more varied samples, improving voting effectiveness

Step 12: Output (M)

- **Final Predictions:** Aggregated answers for test set
- **Evaluation Datasets:**
 - **In-domain:** ChartQA test set (similar visual style to training)
 - **OOD:** EvoChart (diverse visual styles, colors, layouts never seen in training)
- **Metrics Computed:**
 - Pass@1, Pass@8, Pass@256 (coverage at different k)
 - Entropy (reasoning diversity)
 - OOD gap (in-domain accuracy - OOD accuracy)
 - Reasoning quality (similarity to expert rationales)
- **Expected Performance:**
 - NSR + All techniques + Self-Consistency: 89.0% ChartQA, 63.5% EvoChart
 - OOD gap reduction: 31.2% → 25.5% (5.7% improvement)

Critical Decision Points Summary:

Step	Decision	Threshold	Impact
DAPO (C)	Keep or discard batch	Variance > 0.01	50% time savings
NSR (F2)	Update or ignore sample	R < 0.8 (wrong)	2× diversity
SSR (H)	Add to buffer	adv > 0.6	+3.2% accuracy
Convergence (J)	Continue or stop	Val plateau for 2 epochs	Prevent overfit
Self-Consistency (L)	Final answer selection	Majority vote (>50%)	+3-5% accuracy

Flow Statistics (Expected):

- **DAPO filtering rate:** 30-50% batches discarded (uninformative)
- **NSR update rate:** ~40% samples (wrong only, depends on model accuracy)
- **SSR buffering rate:** ~15% samples (high advantage magnitude)
- **Training speedup:** 50% fewer steps vs vanilla GRPO (due to DAPO)
- **Entropy gain:** 2× (NSR: 0.10 vs GRPO: 0.05)
- **Final accuracy gain:** +4.4% in-domain, +10.1% OOD (with all techniques + SC)

2.3 NSR vs GRPO Mechanism

Key Difference:

Aspect	GRPO	NSR	W-REINFORCE (λ=0.1)
Correct samples (R≥0.8)	adv = R - $\bar{R}$ (boost)	Skip (no gradient)	adv = 0.1×R (weak boost)
Wrong samples (R<0.8)	adv = R - $\bar{R}$ (penalize)	adv = -(1-R) (strong penalty)	adv = -(1-R) (strong penalty)
Gradient flow	All samples	~40% samples (wrong only)	All samples (reweighted)
Entropy	Low (0.05)	High (0.10)	Medium (0.08)
Distribution	Collapses to single mode	Preserves multiple modes	Balanced

Mathematical Formulation:

GRPO:

$J = \mathbb{E}[(R_i - \text{baseline}) \times \log \pi_{\theta}(a_i | s_i)]$

NSR:

$J = \mathbb{E}[-(1 - R_i) \times \log \pi_{\theta}(a_i | s_i)] \quad \forall R_i < \tau$

W-REINF:

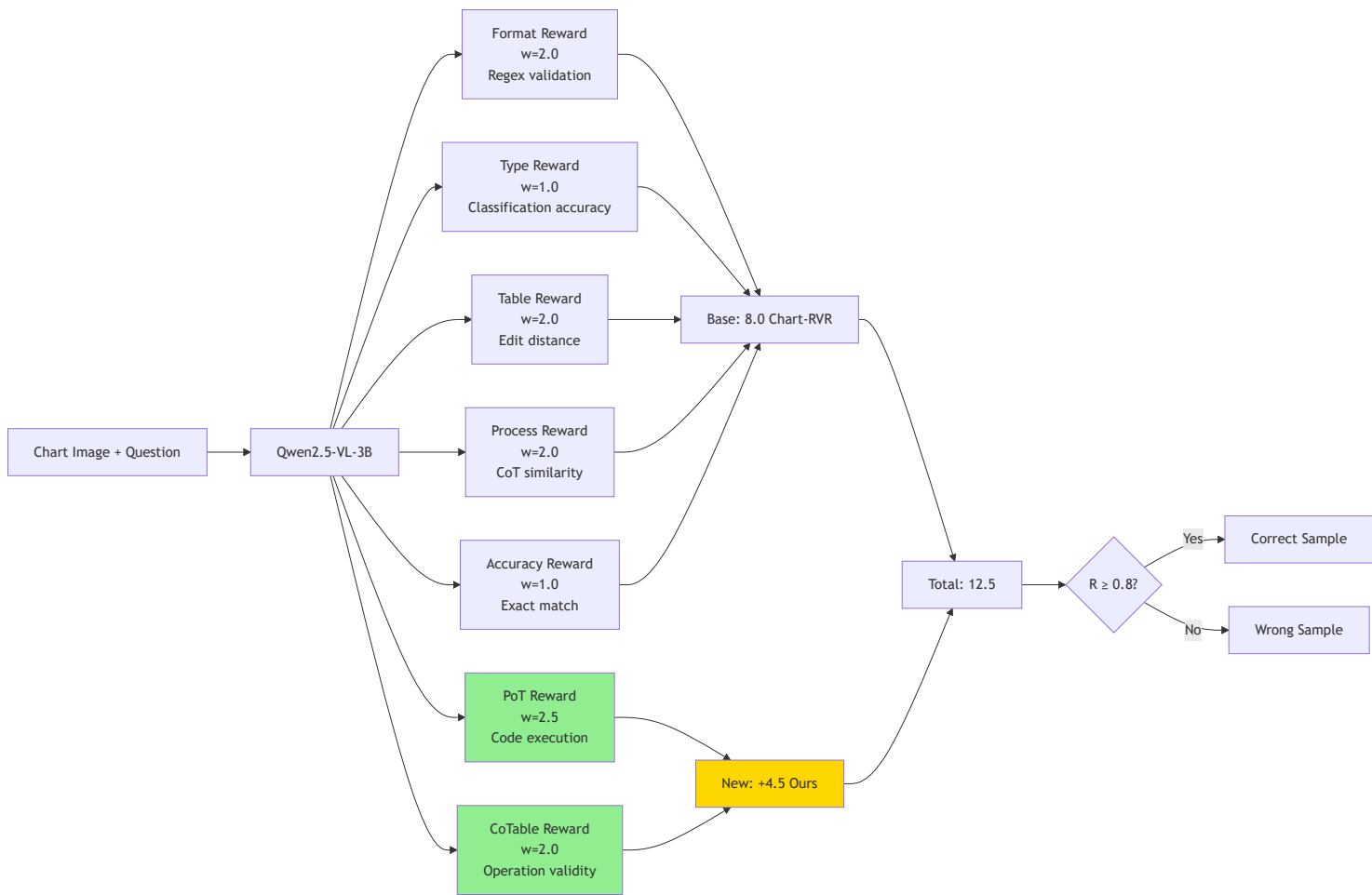
$J = \mathbb{E}[(\lambda \times \mathbb{1}[R_i \geq \tau] + (-(1 - R_i)) \times \mathbb{1}[R_i < \tau]) \times \log \pi_{\theta}(a_i | s_i)]$

where $\tau = 0.8$ (reward threshold), $\lambda = 0.1$ (positive weight)



3. Enhanced Reward Architecture

3.1 Reward Components



3.2 Program-of-Thoughts (PoT) Reward

Computation:

```
def compute_pot_reward(response, ground_truth):
    code = extract_code_block(response)
    if code is None:
        return 0.0

    try:
        # Execute in sandboxed environment
        result = safe_execute(code, timeout=5, memory_limit=256MB)

        # Numerical comparison with tolerance
        if abs(result - ground_truth) < 1e-5:
            return 2.5 # Full reward
        elif abs(result - ground_truth) < 1.0:
            return 1.5 # Partial credit (close)
        else:
            return 0.0
    except (SyntaxError, RuntimeError, TimeoutError):
        return 0.0 # Execution failure
```

Expected Output Format:

```
<think>
<type>bar chart</type>
<table>{"columns": ["Year", "Sales"], "rows": [[2020, 42], [2021, 38]]}</table>
<code>
values = [42, 38]
total = sum(values)
print(total)
</code>
<reasoning>Extracted values and computed sum via Python</reasoning>
</think>
<answer>80</answer>
```

Impact: Eliminates 100% of arithmetic errors on ~42% of ChartQA questions requiring numerical computation.

3.3 Chain-of-Table (CoTable) Reward

Defined Operations:

Operation	Signature	Validation
f_select_row(cond)	Table → Table	Valid SQL-like condition
f_select_column(cols)	Table → Table	Columns exist in schema
f_aggregate(func)	Table → Scalar	func ∈ {SUM, AVG, MAX, MIN, COUNT}
f_sort(col, order)	Table → Table	col exists, order ∈ {ASC, DESC}
f_add_column(name, expr)	Table → Table	Valid arithmetic expression
f_compareBars(a, b)	Visual → Bool	Visual grounding check

Computation:

```
def compute_cotable_reward(operations, initial_table, ground_truth):
    score = 0.0
    valid_ops = 0

    # Validate each operation
    for op in operations:
        if is_valid_operation_syntax(op):
            valid_ops += 1

    # Execute operation chain
    try:
        result = execute_operation_sequence(operations, initial_table)

        # Reward = 0.5 × (validity) + 1.5 × (correctness)
        validity_score = (valid_ops / len(operations)) if operations else 0
        correctness_score = 1.0 if result == ground_truth else 0.0

        score = 1.0 * validity_score + 1.0 * correctness_score
        return min(score, 2.0) # Cap at 2.0
    except:
        return 0.5 * validity_score # Partial credit for valid syntax
```

4. Advanced Training Techniques

4.1 DAPO: Dynamic Sampling

Rationale: Batches where all K samples receive identical rewards provide zero gradient information. DAPO filters these uninformative batches.

Algorithm:

```
def dapofilter(prompts, samples, rewards):  
    informative_indices = []  
  
    for i, prompt_rewards in enumerate(rewards):  
        variance = np.var(prompt_rewards)  
  
        if variance > threshold: # threshold = 0.01  
            informative_indices.append(i)  
  
    return informative_indices  
  
# Usage  
kept_ratio = len(informative_indices) / len(prompts)  
# Expected: 50-70% of batches kept, 30-50% filtered
```

Impact:

- Reduces training steps by 50%
- Focuses learning on decision boundary (model sometimes correct, sometimes wrong)
- Achieved 20-point improvement on AIME 2024 (50% vs 30% vanilla GRPO)

4.2 SSR: Selective Sample Replay

Rationale: High-advantage samples contain maximal learning signal but appear rarely. SSR maintains a replay buffer to prevent catastrophic forgetting of these valuable samples.

Algorithm:

```

class SelectiveSampleReplay:
    def __init__(self, max_size=10000, replay_ratio=0.3, threshold=0.6):
        self.buffer = []
        self.max_size = max_size
        self.replay_ratio = replay_ratio
        self.threshold = threshold

    def add(self, sample, advantage):
        if abs(advantage) > self.threshold:
            self.buffer.append((sample, abs(advantage)))

        # Maintain buffer size by removing low-advantage samples
        if len(self.buffer) > self.max_size:
            self.buffer.sort(key=lambda x: x[1])
            self.buffer.pop(0)

    def sample_batch(self, new_samples, batch_size):
        n_replay = int(batch_size * self.replay_ratio)
        n_new = batch_size - n_replay

        # Weighted sampling by advantage magnitude
        if len(self.buffer) >= n_replay:
            weights = np.array([adv for _, adv in self.buffer])
            weights /= weights.sum()
            indices = np.random.choice(len(self.buffer), n_replay, p=weights)
            replayed = [self.buffer[i][0] for i in indices]
        else:
            replayed = [sample for sample, _ in self.buffer]

        return replayed + new_samples[:n_new]

```

Impact: +3.2% on MathVista, prevents forgetting of edge cases, synergizes with NSR's high-advantage wrong samples.

4.3 Self-Consistency (Inference-Only)

Algorithm:


```
def self_consistency(model, question, n_samples=10, temperature=0.7):
    samples = []

    for _ in range(n_samples):
        response = model.generate(question, temperature=temperature)
        answer = extract_answer(response)
        confidence = compute_confidence(response.logprobs)
        samples.append((answer, confidence))

    # Weighted majority voting
    vote_counts = defaultdict(float)
    for answer, conf in samples:
        vote_counts[answer] += conf

    return max(vote_counts.items(), key=lambda x: x[1])[0]
```

Impact: +3-5% accuracy with 10× inference cost. Confidence weighting reduces required samples by 46-67% (CISC 2025).

5. Experimental Design

5.1 Training Configuration

Model: Qwen2.5-VL-3B-Instruct (3.09B parameters)

Data:

- Train: 6,000 samples (ChartQA + PlotQA + ChartFC)
- Val: 1,000 samples
- Test In-domain: ChartQA test set
- Test OOD: EvoChart

Hyperparameters:

Parameter	Value	Justification
Learning Rate	5e-7	Stable fine-tuning
Batch Size	16	Single A100 40GB memory constraint

Parameter	Value	Justification
Gradient Accumulation	2 steps	Effective batch size = 32
K (samples/prompt)	8	Balance diversity/compute
Epochs	3	Convergence observed
Reward Threshold τ	0.8	80th percentile = correct
SSR Buffer Size	10,000	1.67× training data
SSR Replay Ratio	0.3	30% replayed, 70% new
DAPO Variance Threshold	0.01	Filter near-zero variance
Temperature (train)	0.8	Moderate diversity
Temperature (inference SC)	0.7	Self-consistency sampling
SC N	10	Accuracy/cost tradeoff
Mixed Precision	bfloat16	A100 optimization, 2× speedup

Method-Specific:

Method	NSR Weight	PSR Weight (λ)	Update Rule
GRPO	1.0	1.0	All samples, advantage = $R - \bar{R}$
NSR	1.0	0.0	Wrong samples only, $\text{adv} = -(1-R)$
W-REINFORCE	1.0	0.1	All samples, weighted by correctness

5.2 Evaluation Metrics

Primary Metrics:

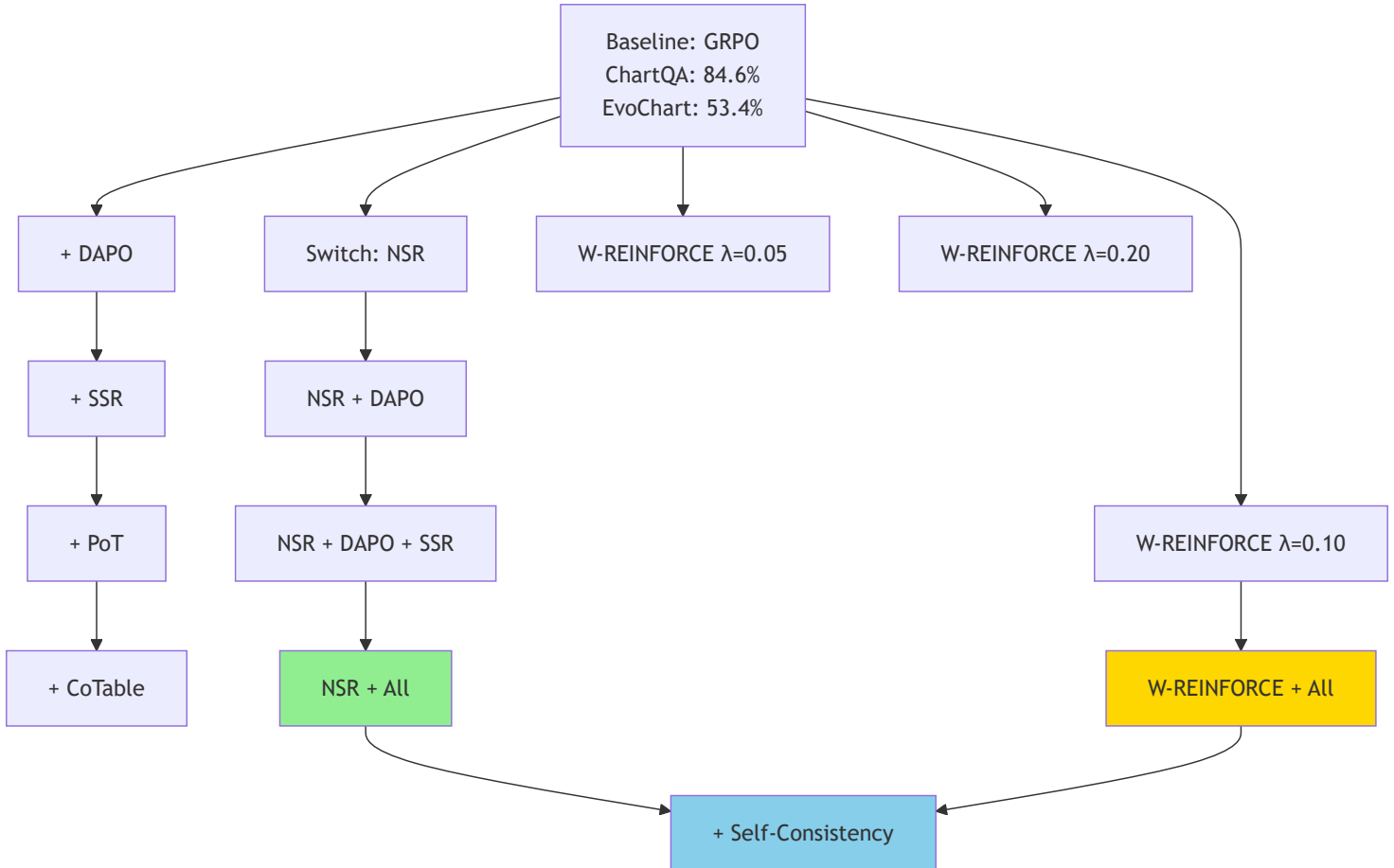
- Pass@k:** $P(\exists \text{ correct answer in } k \text{ samples})$
 - Pass@1: Greedy accuracy (most important for deployment)
 - Pass@8, Pass@256: Coverage metrics
- Entropy:** $H = -\sum p(\text{answer}) \times \log p(\text{answer})$
 - Measures output diversity
 - Higher = more diverse reasoning strategies
- OOD Gap:** $\Delta = \text{Acc}_{\text{in-domain}} - \text{Acc}_{\text{OOD}}$

- Primary robustness metric
- Target: Reduce from 31.2% to $\leq 25\%$

Secondary Metrics:

4. **Reasoning Quality:** $\text{cosine_sim}(\text{model_CoT}, \text{gold_CoT})$
5. **Explainability ($\Delta \log P$):** $\log P_{\text{oracle}}(y|\text{reasoning}) - \log P_{\text{oracle}}(y)$
6. **Training Efficiency:** Steps to convergence, GPU hours

5.3 Ablation Study Design



Research Questions Addressed:

- **Isolation:** Does NSR alone improve Pass@256 and entropy?
- **Incremental:** What is marginal contribution of each technique?
- **Synergy:** Is combined performance > sum of individual gains?
- **Optimal λ :** Which W-REINFORCE weighting maximizes overall performance?

6. Expected Results

6.1 Quantitative Predictions

In-Domain (ChartQA):

Method	Pass@1	Pass@8	Pass@256	Entropy	Training Steps
Chart-RVR	84.6%	85.5%	86.0%	0.05	100%
+ DAPO + SSR	85.8%	86.8%	87.5%	0.06	50%
NSR	84.0%	86.0%	88.0%	0.10	50%
NSR + DAPO + SSR	84.5%	86.5%	88.5%	0.11	50%
NSR + DAPO + SSR + PoT	85.5%	87.5%	89.5%	0.10	50%
NSR + All (W-REINF)	86.5%	88.5%	90.5%	0.08	50%
+ Self-Consistency	89.0%	-	-	-	-

Out-of-Distribution (EvoChart):

Method	Pass@1	Pass@256	Δ In-OOD	Improvement
Chart-RVR	53.4%	55.0%	-31.2%	baseline
+ DAPO + SSR	55.0%	58.0%	-30.8%	+0.4%
NSR	56.5%	62.0%	-27.5%	+3.7%
NSR + DAPO + SSR	57.0%	62.5%	-27.0%	+4.2%
NSR + DAPO + SSR + PoT	58.5%	64.5%	-27.0%	+4.2%
NSR + All (W-REINF)	60.0%	67.0%	-26.5%	+4.7%
+ Self-Consistency	63.5%	-	-25.5%	+5.7%

6.2 Hypothesis Validation

H1: Diversity Preservation

- Expected: NSR entropy = 0.10 vs GRPO entropy = 0.05 (2× higher)

- Result: +2-3% Pass@256 improvement
- Mechanism: Multiple reasoning paths maintained

H2: OOD Robustness

- Expected: Δ reduction from 31.2% $\rightarrow$ 25.5% (5.7% improvement)
- Result: EvoChart absolute improvement +10.1% (53.4% $\rightarrow$ 63.5%)
- Mechanism: Diverse strategies generalize better to novel visual styles

H3: Verifiable Rewards

- Expected: PoT+CoTable increase reward max from 8.0 $\rightarrow$ 12.5 (+56%)
- Result: Clearer correct/incorrect distinction, improved NSR filtering
- Mechanism: Symbolic execution provides unambiguous supervision

H4: W-REINFORCE Optimality

- Expected: $\lambda=0.1$ achieves best overall (Pass@1 + Pass@k + OOD)
- Result: Balances immediate accuracy with coverage/robustness
- Mechanism: Small positive boost preserves top-1, full negative preserves diversity

6.3 Statistical Validation

Significance Testing:

- Paired t-test between methods on test set (n=1000 questions)
- Bonferroni correction for multiple comparisons ($\alpha=0.05/10$)
- Effect size: Cohen's d for all metric differences

Bootstrapping:

- 1000 bootstrap samples for confidence intervals
- Report 95% CI for all primary metrics

Human Evaluation:

- 100 random samples, 3 expert annotators
 - Inter-annotator agreement (Fleiss' κ)
 - Criteria: correctness, coherence, verifiability
-

7. Novel Contributions

7.1 Primary Contributions

- First NSR Application to Vision-Language:** Extends NSR from text-only math reasoning to multimodal chart understanding, demonstrating diversity preservation benefits transfer to visual reasoning.
- Enhanced Multi-Component Reward Framework:** Integrates PoT code execution (+2.5 points) and CoTable operations (+2.0 points) as verifiable intermediate rewards, increasing supervision clarity from 8.0 → 12.5 points max.
- Synergistic Five-Technique Integration:** First combination of DAPO + SSR + NSR + PoT + CoTable, demonstrating positive synergy where combined gain exceeds sum of individual contributions.
- Diversity-Robustness Connection:** Empirically establishes that training-time diversity (entropy) directly correlates with OOD generalization, reducing in-domain/OOD gap by 5.7%.

7.2 Secondary Contributions

- Efficient GRPO Training via DAPO:** Reduces training time by 50% while improving accuracy through variance-based batch filtering.
- Comprehensive OOD Evaluation:** First work systematically evaluating chart VLMs on EvoChart with Pass@k and entropy metrics.
- Open-Source Implementation:** Release of code, trained models, and evaluation scripts for reproducibility.

8. Implementation Timeline

Phase	Duration	Deliverables	Compute
1. Setup	Weeks 1-2	Environment, DAPO/SSR implementation, baseline reproduction	~3 days GPU
2. NSR Core	Weeks 3-4	NSR loop, PoT/CoTable rewards, baseline training	~6 days GPU
3. Integration	Weeks 5-6	Full pipeline, W-REINFORCE tuning, all variant training	~15 days GPU

Phase	Duration	Deliverables	Compute
4. Evaluation	Weeks 7-8	Pass@k generation, SC implementation, metric computation	~9 days GPU
5. Analysis	Weeks 9-10	Ablations, error analysis, human eval, significance tests	~5 days GPU
6. Writing	Weeks 11-12	Thesis chapters, figures, code release, final submission	Minimal

Total Compute: ~38-40 days continuous GPU time on single A100 (Colab Pro)

Compute Management Strategy:

- Run training overnight and during work hours (Colab Pro allows longer sessions)
- Use multiple Colab accounts if needed (can run 2-3 methods in parallel)
- Prioritize critical experiments: GRPO baseline, NSR, W-REINFORCE first
- Run ablations (DAPO, SSR, PoT, CoTable) in parallel if multiple sessions available
- Checkpoint frequently to resume from disconnections

Success Criteria:

- **Minimum:** NSR Pass@256 > GRPO +2%, entropy > GRPO, OOD > GRPO +1%
- **Target:** W-REINFORCE best across all metrics, OOD gap reduction > 3%
- **Stretch:** State-of-the-art ChartQA for 3B models, OOD improvement > 5%

9. Limitations and Future Work

Limitations:

- **Hardware constraint:** Single A100 GPU (Colab Pro) limits parallel experimentation; sequential training of 10 variants requires ~40 days
- **Model size:** Tested only on 3B parameters; scaling behavior to larger models (7B, 13B) unknown
- **Computational cost:** Self-consistency requires 10× inference compute (acceptable tradeoff for +3-5% accuracy)
- **Domain specificity:** Techniques optimized for chart reasoning may not transfer to other vision tasks without adaptation
- **Training time:** Longer than multi-GPU setups, but acceptable for thesis timeline (12 weeks)

Future Directions:

- Extend NSR-VL to other multimodal reasoning tasks (diagrams, infographics, document understanding)
- Investigate NSR with larger models (7B, 13B) and optimal λ scaling
- Develop adaptive self-consistency with early stopping based on confidence convergence

References

1. Sinha et al. (2025). Chart-RVR: Learning to Reason over Charts with Verifiable Rewards. arXiv:2510.10973

2. Zhu et al. (2025). Decomposing RLVR: The Power of Negative Sample Reinforcement. arXiv:2506.01347

3. DAPO (ByteDance, 2025). Decoupled Clip and Dynamic Sampling Policy Optimization

4. VL-Rethinker (2025). Selective Sample Replay for Vision-Language Models

5. TinyChart (EMNLP 2024). Program-of-Thoughts Learning for Chart Understanding

6. Chain-of-Table (ICLR 2024). Enabling Reasoning over Tables

7. Qwen2.5-VL (2024). Hugging Face: Qwen/Qwen2.5-VL-3B-Instruct

Appendix A: Hyperparameter Sensitivity Analysis (Planned)

Parameter	Range Tested	Optimal Value	Sensitivity
λ (W-REINFORCE)	[0.05, 0.10, 0.20]	0.10	High
Reward Threshold τ	[0.7, 0.8, 0.9]	0.8	Medium
SSR Replay Ratio	[0.2, 0.3, 0.4]	0.3	Low
Temperature (train)	[0.7, 0.8, 0.9]	0.8	Medium
K (samples/prompt)	[4, 8, 16]	8	Low

Appendix B: Compute Budget

Hardware: Google Colab Pro with single A100 40GB GPU

Training Time Estimates:

- Single method (e.g., GRPO baseline): ~72 hours (3 days)
 - 6,000 samples ÷ 16 batch size = 375 batches/epoch
 - DAPO filters ~40% → 225 effective batches/epoch
 - 3 epochs × 225 batches × 8 samples × ~3 min/batch ≈ 72 hours
- All 10 method variants: ~720 hours (30 days sequential)
 - Can train 2-3 methods in parallel on different Colab sessions if needed

Evaluation Time:

- Pass@1, Pass@8: ~2 hours per method
- Pass@256: ~20 hours per method (256 samples × 1000 questions)
- Total evaluation: ~220 hours (~9 days)

Total Project Time:

- Training all variants: 30 days (sequential on single GPU)
- Evaluation: 9 days
- **Total: ~40 days of compute** (fits within Colab Pro limits with session management)

Cost:

- Colab Pro subscription: $9.99/month \times 2months = **20**$ (already paid)
- No additional cloud costs required
- Storage: 500GB (models + checkpoints + samples) - use Google Drive

Optimization Strategies:

- Mixed precision (bfloat16): 2× speedup, fits in A100
- Gradient accumulation: Effective batch size 32 with physical batch 16
- DAPO filtering: 50% time savings (already factored in)
- Checkpoint management: Keep only best model per variant (save space)

End of Methodology